

Лекция 4: классы типов

Функциональное программирование на Haskell

Алексей Романов

4 сентября 2026 г.

МИЭТ

Классы типов

- Полиморфные функции, которые мы видели в прошлый раз, имеют одно определение для любых параметров типов.
- Часто нужно определить функции по-разному для разных типов.
- Для этого используются классы типов.

<MINTED>

- **Eq** — название класса.
- **a** — тип, относящийся к этому классу.
- **<MINTED>** и **<MINTED>** — функции-члены класса, с определениями по умолчанию.
- Их нужно определить для типа **a**

Классы типов

- Полиморфные функции, которые мы видели в прошлый раз, имеют одно определение для любых параметров типов.
- Часто нужно определить функции по-разному для разных типов.
- Для этого используются классы типов.

<MINTED>

- **Eq** — название класса.
- **a** — тип, относящийся к этому классу.
- **<MINTED>** и **<MINTED>** — функции-члены класса, с определениями по умолчанию.
- Их нужно определить для типа **a** (достаточно одну).

- Чтобы объявить, что тип `Bool` относится к классу `Eq`:
`<MINTED>`
- При этом `a` в сигнатуре членов заменяется на тип, для которого определяется экземпляр.
- Ещё пример:

- Чтобы объявить, что тип `Bool` относится к классу `Eq`:
`<MINTED>`
- При этом `a` в сигнатуре членов заменяется на тип, для которого определяется экземпляр.
- Ещё пример:
`<MINTED>`

- Чтобы объявить, что тип `Bool` относится к классу `Eq`:
`<MINTED>`
- При этом `a` в сигнатуре членов заменяется на тип, для которого определяется экземпляр.
- Ещё пример:
`<MINTED>`
- Рекурсивно ли это определение?

- Чтобы объявить, что тип `Bool` относится к классу `Eq`:
`<MINTED>`
- При этом `a` в сигнатуре членов заменяется на тип, для которого определяется экземпляр.
- Ещё пример:
`<MINTED>`
- Это не рекурсия: `==` в правой части относится к другому типу (`String`).

Ограниченный полиморфизм

- Если спросить GHCi про тип `==`, получим **<MINTED>**
- Читается «**<MINTED>** для любого `a` из `Eq`».
- Часть перед **<MINTED>** называется контекстом.
- Его элементы — ограничения. Когда их больше одного, они пишутся в скобках через запятую.

Ограниченный полиморфизм

- Если спросить GHCi про тип `==`, получим **<MINTED>**
- Читается «**<MINTED>** для любого `a` из `Eq`».
- Часть перед **<MINTED>** называется контекстом.
- Его элементы — ограничения. Когда их больше одного, они пишутся в скобках через запятую.
- Типы полиморфных функций, *использующих* `==` и `/=`, пишутся аналогично. Определим:

<MINTED>

Какой у неё тип (и смысл)?

Ограниченный полиморфизм

- Если спросить GHCi про тип `==`, получим **<MINTED>**
- Читается «**<MINTED>** для любого `a` из `Eq`».
- Часть перед **<MINTED>** называется контекстом.
- Его элементы — ограничения. Когда их больше одного, они пишутся в скобках через запятую.
- Типы полиморфных функций, *использующих* `==` и `/=`, пишутся аналогично. Определим:
<MINTED>
Какой у неё тип (и смысл)?
- **<MINTED>**

Ограниченный полиморфизм

- Если спросить GHCi про тип `==`, получим **<MINTED>**
- Читается «**<MINTED>** для любого `a` из `Eq`».
- Часть перед **<MINTED>** называется контекстом.
- Его элементы — ограничения. Когда их больше одного, они пишутся в скобках через запятую.
- Типы полиморфных функций, *использующих* `==` и `/=`, пишутся аналогично. Определим:
<MINTED>
Какой у неё тип (и смысл)?
- **<MINTED>**
- Функция проверяет, равны ли все элементы списка.

Экземпляры для полиморфных типов

- Попробуем определить `Eq` для `Maybe a`:

Экземпляры для полиморфных типов

- Попробуем определить `Eq` для `Maybe a`:
<MINTED>
- Верное ли это определение?

Экземпляры для полиморфных типов

- Попробуем определить `Eq` для `Maybe a`:
<MINTED>
- Здесь `a` не обязательно `Eq`, и поэтому **<MINTED>**
не скомпилируется.
- А можно ли так?
<MINTED>

Экземпляры для полиморфных типов

- Попробуем определить `Eq` для `Maybe a`:
<MINTED>
- Здесь `a` не обязательно `Eq`, и поэтому **<MINTED>** не скомпилируется.
- А можно ли так?
<MINTED>
- Нет: эта сигнатура для **<MINTED>** более ограничена, чем заданная классом.

Экземпляры для полиморфных типов с ограничениями

- Правильно так:
<MINTED>
- Читаем «**Maybe** **a** относится к **Eq** только тогда, когда **a** относится к **Eq**».
- Контексты экземпляров подчиняются тем же правилам, что контексты функций.
- Ограничения могут упоминать другие классы, а не только тот, экземпляр которого определяется.
- Не может быть много экземпляров одного класса для одного типа с разными условиями.

Аналоги в других языках

- Как перегрузка функций (или операторов), но глубже встроенная в систему типов.
- Перегруженные функции — просто разные функции, связанные только общим именем.
- Здесь обязательно близкие типы и общий смысл.
- И функции могут быть использованы другими полиморфными функциями.

Аналоги в других языках

- Как перегрузка функций (или операторов), но глубже встроенная в систему типов.
- Перегруженные функции — просто разные функции, связанные только общим именем.
- Здесь обязательно близкие типы и общий смысл.
- И функции могут быть использованы другими полиморфными функциями.
- Не путайте классы типов и классы в ООП, общее только название!
- В C++ есть близкое понятие концепций.

Аналоги в других языках

- Как перегрузка функций (или операторов), но глубже встроенная в систему типов.
- Перегруженные функции — просто разные функции, связанные только общим именем.
- Здесь обязательно близкие типы и общий смысл.
- И функции могут быть использованы другими полиморфными функциями.
- Не путайте классы типов и классы в ООП, общее только название!
- В C++ есть близкое понятие концепций.
- До C++20 было только для документации, а теперь их можно объявлять и использовать в коде.

Аналоги в других языках

- Как перегрузка функций (или операторов), но глубже встроенная в систему типов.
- Перегруженные функции — просто разные функции, связанные только общим именем.
- Здесь обязательно близкие типы и общий смысл.
- И функции могут быть использованы другими полиморфными функциями.
- Не путайте классы типов и классы в ООП, общее только название!
- В C++ есть близкое понятие концепций.
- До C++20 было только для документации, а теперь их можно объявлять и использовать в коде.
- Часто сравнивают с интерфейсами в Java/C#, но это подходит хуже:

Аналоги в других языках

- Как перегрузка функций (или операторов), но глубже встроенная в систему типов.
- Перегруженные функции — просто разные функции, связанные только общим именем.
- Здесь обязательно близкие типы и общий смысл.
- И функции могут быть использованы другими полиморфными функциями.
- Не путайте классы типов и классы в ООП, общее только название!
- В C++ есть близкое понятие концепций.
- До C++20 было только для документации, а теперь их можно объявлять и использовать в коде.
- Часто сравнивают с интерфейсами в Java/C#, но это подходит хуже: в классах типов есть «статические» и бинарные методы, совсем другой смысл функции, возвращающей такой тип.

Другие важные классы

- Линейно упорядоченные типы:

<MINTED>

- **<MINTED>** — надкласс **<MINTED>**, то есть любой экземпляр **<MINTED>** должен быть и экземпляром **Eq**.

Другие важные классы

- Линейно упорядоченные типы:

<MINTED>

- **<MINTED>** — надкласс **<MINTED>**, то есть любой экземпляр **<MINTED>** должен быть и экземпляром **Eq**.
- Логически стрелка скорее в другом направлении:
a относится к **<MINTED>** $\implies$ **a** относится к **Eq**.

Другие важные классы

- Линейно упорядоченные типы:

<MINTED>

- **<MINTED>** — надкласс **<MINTED>**, то есть любой экземпляр **<MINTED>** должен быть и экземпляром **Eq**.
- Логически стрелка скорее в другом направлении:
a относится к **<MINTED>** $\implies$ **a** относится к **Eq**.

- Типы, ограниченные сверху и снизу:

<MINTED>

- Это не подкласс **<MINTED>** потому, что порядок, для которого эти значения — границы, может быть частичным.

Преобразования в строки и обратно

- Типы, значения которых можно превратить в строки:
<MINTED>

Преобразования в строки и обратно

- Типы, значения которых можно превратить в строки:
<MINTED>
- Или прочитать из строк:
<MINTED>

Преобразования в строки и обратно

- Типы, значения которых можно превратить в строки:
<MINTED>
- Или прочитать из строк:
<MINTED>
- Для нас то, что за многоточиями, неважно, но подробности описаны **в документации**.

Преобразования в строки и обратно

- Типы, значения которых можно превратить в строки:
<MINTED>
- Или прочитать из строк:
<MINTED>
- Для нас то, что за многоточиями, неважно, но подробности описаны **в документации**.
- **<MINTED>** должна быть обратной к **show**.
- Заметьте полиморфизм **<MINTED>** по возвращаемому типу:
<MINTED>

Опасность <MINTED>

- Что случится, если передать <MINTED> неподходящий аргумент?
<MINTED>
<MINTED>
- Если вы не знаете *точно*, что текст содержит значение, используйте безопасную
<MINTED>
из модуля <MINTED>.

Опасность <MINTED>

- Что случится, если передать <MINTED> неподходящий аргумент?
<MINTED>
<MINTED>
- Если вы не знаете *точно*, что текст содержит значение, используйте безопасную
<MINTED>
из модуля <MINTED>.
- Это не единственная функция в Prelude с такой проблемой.
- Пакет **safe** содержит безопасные варианты для всех таких функций (посмотрите его документацию).

- **<MINTED>** содержит типы, значения которых образуют последовательность.
<MINTED>

- **<MINTED>** содержит типы, значения которых образуют последовательность.

<MINTED>

- Для него есть специальный синтаксис:
 - **<MINTED>** означает **<MINTED>**;
 - **<MINTED>** — **<MINTED>**;
 - **<MINTED>** — **<MINTED>**;
 - **<MINTED>** — **<MINTED>**.

Класс перечислимых типов

- **<MINTED>** содержит типы, значения которых образуют последовательность.
<MINTED>
- Для него есть специальный синтаксис:
 - **<MINTED>** означает **<MINTED>**;
 - **<MINTED>** — **<MINTED>**;
 - **<MINTED>** — **<MINTED>**;
 - **<MINTED>** — **<MINTED>**.
- Для **Float** и **Double** есть экземпляры **<MINTED>**, так что можно писать **<MINTED>**:
<MINTED>

Класс перечислимых типов

- **<MINTED>** содержит типы, значения которых образуют последовательность.

<MINTED>

- Для него есть специальный синтаксис:
 - **<MINTED>** означает **<MINTED>**;
 - **<MINTED>** — **<MINTED>**;
 - **<MINTED>** — **<MINTED>**;
 - **<MINTED>** — **<MINTED>**.
- Для **Float** и **Double** есть экземпляры **<MINTED>**, так что можно писать **<MINTED>**, но они довольно странные:

<MINTED>

Иерархия числовых классов

- Вернёмся к «числовым типам» из первой лекции.
- Это тоже классы типов, образующие довольно сложную иерархию.

Иерархия числовых классов

- Вернёмся к «числовым типам» из первой лекции.
- Это тоже классы типов, образующие довольно сложную иерархию.

<MINTED>

Иерархия числовых классов: дробные типы

<MINTED>

- Полная документация.

Числовые литералы

- Литерал `0` означает `<MINTED>` и соответственно получает тип

Числовые литералы

- Литерал `0` означает `<MINTED>` и соответственно получает тип `<MINTED>`.
- Аналогично для дробных: `<MINTED>` представляется как рациональное и превращается в `<MINTED>` типа `<MINTED>`.
- Здесь:
`<MINTED>`

Числовые литералы

- Литерал `0` означает **<MINTED>** и соответственно получает тип **<MINTED>**.
- Аналогично для дробных: **<MINTED>** представляется как рациональное и превращается в **<MINTED>** типа **<MINTED>**.
- Здесь:
<MINTED>
- Использование рациональных чисел для дробных литералов позволяет избежать ошибок округления.

Числовые литералы

- Литерал `0` означает **<MINTED>** и соответственно получает тип **<MINTED>**.
- Аналогично для дробных: **<MINTED>** представляется как рациональное и превращается в **<MINTED>** типа **<MINTED>**.
- Здесь:
<MINTED>
- Использование рациональных чисел для дробных литералов позволяет избежать ошибок округления.
- Конечно, при преобразовании в `Float` или `Double` они вернутся.

Алгебраические классы

- Для разных структур из общей алгебры или линейной алгебры можно определить соответствующие им классы. В `Prelude` есть
- полугруппы (ассоциативная операция):
`<MINTED>`
- и моноиды (полугруппа с нейтральным элементом):
`<MINTED>`
- Так стало с GHC 8.4; в старом коде и книгах
`<MINTED>` — основной метод, а `<MINTED>` нет вообще.

Алгебраические классы

- Для разных структур из общей алгебры или линейной алгебры можно определить соответствующие им классы. В `Prelude` есть
- полугруппы (ассоциативная операция):
`<MINTED>`
- и моноиды (полугруппа с нейтральным элементом):
`<MINTED>`
- Так стало с GHC 8.4; в старом коде и книгах `<MINTED>` — основной метод, а `<MINTED>` нет вообще.
- Ещё несколько из теории категорий, но это предмет будущих лекций.

Определение функций внутри или вне класса

- При определении функций над классом часто приходится выбирать из двух вариантов:
 - член класса с определением по умолчанию;
 - функция, определённая вне класса.
- Например, **<MINTED>** можно было бы определить вне класса:

<MINTED>

Определение функций внутри или вне класса

- При определении функций над классом часто приходится выбирать из двух вариантов:
 - член класса с определением по умолчанию;
 - функция, определённая вне класса.
- Например, **<MINTED>** можно было бы определить вне класса:
<MINTED>
- Такую функцию нельзя реализовать по-разному для разных экземпляров.

Определение функций внутри или вне класса

- При определении функций над классом часто приходится выбирать из двух вариантов:
 - член класса с определением по умолчанию;
 - функция, определённая вне класса.
- Например, **<MINTED>** можно было бы определить вне класса:
<MINTED>
- Такую функцию нельзя реализовать по-разному для разных экземпляров.
- Если для каких-то типов есть реализация лучше той, что по умолчанию, лучше сделать функцию членом класса.

Определение функций внутри или вне класса

- При определении функций над классом часто приходится выбирать из двух вариантов:
 - член класса с определением по умолчанию;
 - функция, определённая вне класса.
- Например, **<MINTED>** можно было бы определить вне класса:
<MINTED>
- Такую функцию нельзя реализовать по-разному для разных экземпляров.
- Если для каких-то типов есть реализация лучше той, что по умолчанию, лучше сделать функцию членом класса.
 - Очевидная проблема: вы можете не знать (или не подумать) о таких типах.

Определение функций внутри или вне класса

- При определении функций над классом часто приходится выбирать из двух вариантов:
 - член класса с определением по умолчанию;
 - функция, определённая вне класса.
- Например, **<MINTED>** можно было бы определить вне класса:
<MINTED>
- Таковую функцию нельзя реализовать по-разному для разных экземпляров.
- Если для каких-то типов есть реализация лучше той, что по умолчанию, лучше сделать функцию членом класса.
 - Очевидная проблема: вы можете не знать (или не подумать) о таких типах.
- Или если реализация по умолчанию не всегда верна

Определение функций внутри или вне класса

- При определении функций над классом часто приходится выбирать из двух вариантов:
 - член класса с определением по умолчанию;
 - функция, определённая вне класса.
- Например, **<MINTED>** можно было бы определить вне класса:

<MINTED>

- Такую функцию нельзя реализовать по-разному для разных экземпляров.
- Если для каких-то типов есть реализация лучше той, что по умолчанию, лучше сделать функцию членом класса.
 - Очевидная проблема: вы можете не знать (или не подумать) о таких типах.
- Или если реализация по умолчанию не всегда верна (но тогда почему это реализация по умолчанию?).

- Рассмотрим такой экземпляр типа:
<MINTED>
- Даже не зная смысла **<MINTED>** мы видим, что что-то здесь не так. Что?

Законы классов типов

- Рассмотрим такой экземпляр типа:
<MINTED>
- Даже не зная смысла **<MINTED>** мы видим, что что-то здесь не так.
- Это определение нерефлексивно, то есть **<MINTED>**.
- У всех классов типов (почти) есть законы.
- То есть наличия функций типов, указанных в определении, недостаточно;
- для корректных экземпляров должны выполняться определённые законы.

- Например, для `Eq` законы такие:

- Например, для `Eq` законы такие:
`<MINTED>`

Законы классов типов

- Например, для `Eq` законы такие:
`<MINTED>`
- Законы могут говорить и о связи между классами.
Например, если тип одновременно `<MINTED>` и
`<MINTED>`, то естественно требовать
`<MINTED>`

Законы классов типов

- Например, для `Eq` законы такие:
`<MINTED>`
- Законы могут говорить и о связи между классами. Например, если тип одновременно `<MINTED>` и `<MINTED>`, то естественно требовать `<MINTED>`
- Компилятор не может проверить законы.
- Поэтому ответственность за их соблюдение лежит на программисте.

Автоматический вывод экземпляров

- Почти все определения **<MINTED>** устроены одинаково:

Автоматический вывод экземпляров

- Почти все определения **<MINTED>** устроены одинаково:
 - Типы всех полей всех конструкторов должны быть **Eq**;
 - Значения с одинаковым конструктором равны, если равны все их поля (или полей нет);
 - Значения с разными конструкторами не равны;
- Почти, но не все:

Автоматический вывод экземпляров

- Почти все определения **<MINTED>** устроены одинаково:
 - Типы всех полей всех конструкторов должны быть **Eq**;
 - Значения с одинаковым конструктором равны, если равны все их поля (или полей нет);
 - Значения с разными конструкторами не равны;
- Почти, но не все: например, для **Data.Set** и **Data.Map** (два сбалансированных дерева с одними и теми же элементами могут иметь разную форму) или для типов с кэшированными полями.

Автоматический вывод экземпляров

- Почти все определения **<MINTED>** устроены одинаково:
 - Типы всех полей всех конструкторов должны быть **Eq**;
 - Значения с одинаковым конструктором равны, если равны все их поля (или полей нет);
 - Значения с разными конструкторами не равны;
- Почти, но не все: например, для **Data.Set** и **Data.Map** (два сбалансированных дерева с одними и теми же элементами могут иметь разную форму) или для типов с кэшированными полями.
- Чтобы не писать такие определения каждый раз, после **data** или **newtype** ставят **<MINTED>**.
- **deriving** также работает для **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>** и **<MINTED>**.
- Порядок на значениях для **<MINTED>**, **<MINTED>** и **<MINTED>** задаётся порядком конструкторов в определении.

Автоматический вывод экземпляров

- Почти все определения **<MINTED>** устроены одинаково:
 - Типы всех полей всех конструкторов должны быть **Eq**;
 - Значения с одинаковым конструктором равны, если равны все их поля (или полей нет);
 - Значения с разными конструкторами не равны;
- Почти, но не все: например, для **Data.Set** и **Data.Map** (два сбалансированных дерева с одними и теми же элементами могут иметь разную форму) или для типов с кэшированными полями.
- Чтобы не писать такие определения каждый раз, после **data** или **newtype** ставят **<MINTED>**.
- **deriving** также работает для **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>** и **<MINTED>**.
- Порядок на значениях для **<MINTED>**, **<MINTED>** и **<MINTED>** задаётся порядком конструкторов в определении.
- Разные расширения GHC позволяют **deriving** для

- Для каждой пары класса и типа можно определить только один экземпляр.
- А что делать, если осмысленных экземпляров больше одного?
- Например, числа — моноиды по операциям $+$ и $*$.

Классы типов и `newtype`

- Для каждой пары класса и типа можно определить только один экземпляр.
- А что делать, если осмысленных экземпляров больше одного?
- Например, числа — моноиды по операциям $+$ и $*$.
- В таком случае мы можем завернуть существующий тип в `newtype` и определить для нового:

`<MINTED>`

Классы типов и `newtype`

- Если заметили, `<MINTED>` не было в списке классов, для которых есть автоматический вывод экземпляров.
- Для `newtype` можно вывести любой класс с помощью расширения `<MINTED>`.

- Если заметили, `<MINTED>` не было в списке классов, для которых есть автоматический вывод экземпляров.
- Для `newtype` можно вывести любой класс с помощью расширения `<MINTED>`.
- Выведенное определение каждой функции вызывает ту же функцию завёрнутого типа с вставкой и разбором конструкторов где нужно:
`<MINTED>`

Классы типов и модули

- **<MINTED>** в списке импорта/экспорта означает класс со всеми методами, как для типа.
- Поскольку экземпляры не имеют имён, они экспортируются из модуля всегда.
- И импортируются всегда, когда модуль упоминается в списке импорта.

Классы типов и модули

- **<MINTED>** в списке импорта/экспорта означает класс со всеми методами, как для типа.
- Поскольку экземпляры не имеют имён, они экспортируются из модуля всегда.
- И импортируются всегда, когда модуль упоминается в списке импорта.
- Из-за этого существование экземпляров для пары класс-тип не в одном модуле — проблема.
- Поэтому экземпляры обычно определяются либо в том модуле, где определён тип, либо в том, где определён класс.
- Остальные называются сиротами и по возможности их создания следует избегать.
- Это делается с помощью того же трюка с **newtype**.

Ограничения классов типов в стандартном Haskell и расширения GHC

- В стандартном Haskell есть несколько ограничений на объявления классов типов и экземпляров.
- А в GHC расширений, которые их снимают.
- Все перечислять не будем, они есть в [User's Guide \(Instance declarations and resolution\)](#).
- Сейчас достаточно знать, что если что-то не компилируется из-за отсутствия **<MINTED>**, **<MINTED>** или **<MINTED>**, их спокойно можно включить.
- **<MINTED>** и прагмы **<MINTED>/<MINTED>** у отдельных экземпляров (расширения **<MINTED>/<MINTED>** устарели с GHC 7.10) опаснее, но в этом курсе в любом случае не понадобятся.